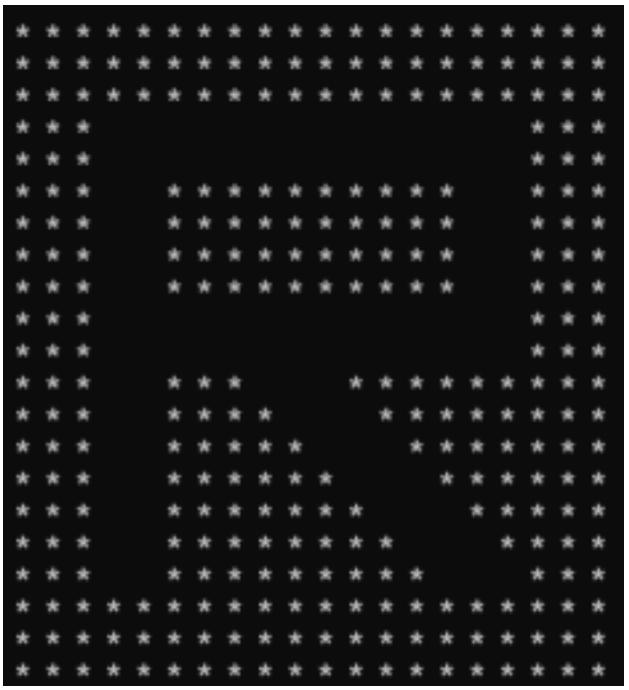
DEN Coding Competition `24

Rules:

- **Timing**: Participants have **2 hours** to solve the problem, with an additional **15 minutes** for submission.
- **Submission Format**: Ensure code is properly commented and organized. • **Scoring**: Solutions are evaluated based on accuracy and adherence to specified dimensions and character requirements.

Problem # 01

Using a combination of asterisks ('*') and spaces (' '), create the letter 'R' within dimensions 21 rows by 20 columns (21 X 20). Ensure the letter 'R' is precisely represented as follows.



Note: Solve this problem using conditions and only "two" for loops

Problem # 02

In a distributed computing environment, multiple servers across different geographical locations generate logs related to user activities. These logs are stored in various files and must be analysed to extract insights about user behaviours across different time zones. The logs are stored daily and named based on the date and server identifier. Your task is to create a system that processes these logs to identify anomalies such as sudden surges in activity and potential security threats based on predefined patterns.

Task:

Build a multi-threaded application that reads log files from multiple servers stored over multiple days, processes them concurrently, and identifies and logs specific patterns of activities defined in a configuration file.

Input:

- A configuration file (**config.txt**) that contains:
    - Patterns to search for in the logs.
    - The threshold for each pattern that defines an anomaly.
    - The server and its corresponding timezone offset.
- A directory structure where each subdirectory is named after a server and contains log files named in the format **YYYY-MM-DD.log**.

Output:

- A report file (**report.txt**) listing all identified anomalies, including the server, date, pattern, and count.

Sample Configuration File (config.txt):

Pattern1: login failed, 5

Pattern2: transaction exceeded, 3

Server1: -5

Server2: +3

Sample Log Entry (2021-06-01.log in the Server1 directory):

2021-06-01T08:00:00 login failed

2021-06-01T08:05:00 login failed

2021-06-01T08:15:00 transaction exceeded $5000

2021-06-01T08:20:00 login failed

Expected Output in report.txt:

2021-06-01, Server1, Pattern1, 3 times

2021-06-01, Server1, Pattern2, 1 time

Problem # 03

The task involves solving a variation of the Traveling Salesman, which is to find the shortest possible route that visits each city exactly once and returns to the starting point. Your challenge here includes not just determining the route between cities but also choosing the optimal route from multiple options between each pair of cities. The given problem is further complicated by the presence of multiple routes between cities with different distances.

Let's break down the problem:

1. **Cities and Routes**: The salesman starts from Islamabad and has to visit Lahore, Gujranwala, and Karachi before returning to Islamabad.

2. **Distances**: Multiple distances are given between each pair of cities, and we need to pick the shortest one for our calculation.

3. **Objective**: Minimize the total distance traveled.

You are required to create a 3D matrix to represent distances between cities. The dimensions of the matrix will be 3 cities × 3 cities × 5 possible routes, reflecting the multiple route options between each pair of cities.

Creating the 3D Matrix

The matrix will be indexed such that:

- The first dimension corresponds to the source city.
- The second dimension corresponds to the destination city.
- The third dimension corresponds to the different routes available between the source and destination.

For simplification, let's index the cities as follows:

- 0: Islamabad
- 1: Lahore
- 2: Gujranwala
- 3: Karachi

The 3D matrix (**distances**) can be visualized as follows (in Python-like pseudo-code): To present the distance data between cities in a more structured and readable format, you can use a series of tables for each pair of cities, showcasing the different routes available. This will help visualize the 3D matrix concept in a traditional tabular format.

Distances from Islamabad to Other Cities

Here's how the distances are structured from Islamabad to Lahore, Gujranwala, and Karachi:

From	To	Route 1	Route 2	Route 3	Route 4	Route 5
Islamabad	Islamabad	0	0	0	0	0
Islamabad	Lahore	300	300	300	300	300
Islamabad	Gujranwala	250	250	250	250	250
Islamabad	Karachi	500	500	500	500	500

Distances from Lahore to Other Cities

Here are the distances from Lahore to each other city, including itself (for intra-city routes):

From	To	Route 1	Route 2	Route 3	Route 4	Route 5
Lahore	Islamabad	300	300	300	300	300
Lahore	Lahore	20	38	51	11	8
Lahore	Gujranwala	89	92	101	42	39
Lahore	Karachi	150	172	158	111	62

Distances from Gujranwala to Other Cities

The distances from Gujranwala to each other city are as follows:

From	To	Route 1	Route 2	Route 3	Route 4	Route 5
Gujranwala	Islamabad	250	250	250	250	250
Gujranwala	Lahore	89	92	101	42	39
Gujranwala	Gujranwala	11	15	9	2	4
Gujranwala	Karachi	69	71	77	89	85

Distances from Karachi to Other Cities

Finally, the distances from Karachi to each of the cities are detailed below:

From	To	Route 1	Route 2	Route 3	Route 4	Route 5
Karachi	Islamabad	500	500	500	500	500
Karachi	Lahore	150	172	158	111	62
Karachi	Gujranwala	69	71	77	89	85
Karachi	Karachi	22	28	59	51	72

Sample Input and Output

Input	Output

Please enter the starting city: Islamabad Enter the list of cities you wish to visit separated by commas (e.g., Lahore, Gujranwala, Karachi): Lahore, Gujranwala	Calculating the shortest possible route starting and ending at Islamabad... Optimal Route: Islamabad -> Lahore -> Gujranwala -> Karachi -> Islamabad Minimum Distance: 932 km Thank you for using the City Route Planner. Safe travels!
Please enter the starting city: Islamabad Enter the list of cities you wish to visit separated by commas (e.g., Lahore, Gujranwala, Karachi): Lahore, Karachi	Calculating the shortest possible route starting and ending at Islamabad... Optimal Route: Islamabad -> Lahore -> Karachi -> Islamabad Minimum Distance: 862 km Thank you for using the City Route Planner. Safe travels!
Please enter the starting city: Islamabad Enter the list of cities you wish to visit separated by commas (e.g., Lahore, Gujranwala, Karachi): Gujranwala, Karachi	Calculating the shortest possible route starting and ending at Islamabad... Optimal Route: Islamabad -> Gujranwala -> Karachi -> Lahore -> Islamabad

	Minimum Distance: 929 km Thank you for using the City Route Planner. Safe travels!

Problem # 04

A hybrid data structure that combines a hash table and a doubly linked list to efficiently manage a caching system known as Least Recently Used (LRU) Cache. This system must efficiently provide fast access to the most recently used items while maintaining a fixed capacity by discarding the least recently used items when necessary.

LRU Cache Mechanics: Items in the cache should be accessed in constant time. When an item is accessed or added, it becomes the most recently used item. If the cache exceeds its capacity when adding a new item, the least recently used item must be removed.

Implementation Guidelines:

- **Hash Table:** For fast access to the nodes of the doubly linked list, ensuring O(1) time complexity for cache operations.
- **Doubly Linked List:** To keep the items in the order of use. The front of the list contains the most recently used item, while the rear of the list holds the least recently used item.
- **Operations:**
    - **get(key)**: Returns the value of the key if the key exists in the cache, otherwise returns -1. This operation should move the item to the front of the list, marking it as most recently used.
    - **put(key, value)**: Inserts or replaces the value if the key is already present. If the cache has reached its capacity, the least recently used item is removed before a new item is added.
    - **display()**: Prints the current items in the cache from most to least recently used.

Constraints:

- All operations must run in O(1) time.
- The number of nodes in the linked list is equal to the number of items in the hash table.

Sample Input and Output:

Console Input:

Initialize cache with capacity: 3

Put 1, "Data1"

Put 2, "Data2"

Get 1

Put 3, "Data3"

Put 4, "Data4"

Get 2

Display

Console Output:

// Outputs after operations

Initialized cache with capacity 3

Put 1: Cache = ["Data1"]

Put 2: Cache = ["Data2", "Data1"]

Get 1: "Data1" -> Cache updated to ["Data1", "Data2"]

Put 3: Cache = ["Data3", "Data1", "Data2"]

Put 4: Cache = ["Data4", "Data3", "Data1"] // "Data2" removed because it was the least recently used

Get 2: -1 // "Data2" is not in the cache

Display: ["Data4", "Data3", "Data1"]